Project Title: Privacy-Preserving Offline Natural Language Interface for OS Control Using Small Language Models

1. Executive Summary

During Week 1, the primary focus was formalizing the project scope, defining the risk mitigation strategy for local OS control, and conducting a literature review based on two foundational research papers regarding Small Language Model (SLM) function-calling capabilities and adapter-based modular architectures.

2. Literature Review & Technical Justification

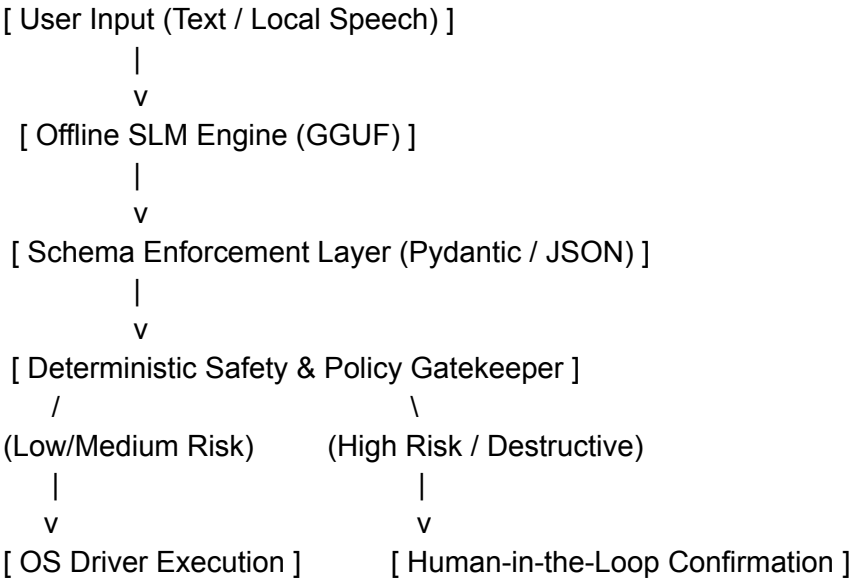
Paper 1: *Small Models, Big Tasks: An Exploratory Empirical Study on Small Language Models for Function Calling* (Kavathekar et al., 2025)

- **Edge & Memory Efficiency:** Sub-4B parameter SLMs quantized to 4-bit GGUF format reduce memory footprints by ~5x compared to full-precision server models, proving that local execution on consumer hardware is practical.
- **Formatting Breakdown:** Unconstrained zero-shot SLMs struggle significantly with strict JSON schema adherence. However, fine-tuning or schema-constrained decoding elevates JSON parsability from 7.34% to over 99%.
- **Prompt Injection Resilience:** SLMs show strong baseline resilience against direct prompt injection perturbations, maintaining structural integrity with minimal performance loss.
- **System Implication:** Validates our plan to use an offline GGUF inference engine paired with schema-constrained decoding (e.g., Pydantic) to eliminate syntax errors and hallucinated commands.

Paper 2: *SLM: Bridge the Thin Gap Between Speech and Text Foundation Models* (Wang et al., Google DeepMind)

- **Lightweight Adaptation:** Demonstrates that a frozen foundation model paired with a small, trainable adapter (comprising ~1% of total model parameters) can effectively map input representations into structured outputs without retraining the underlying base model.
- **Instruction Following:** Proves that thin adaptation layers preserve the core instruction-following capabilities of the base model while enabling specialized downstream tasks.
- **System Implication:** Supports our modular architecture where the base SLM remains untouched, and system safety/tool execution logic is handled by lightweight, deterministic outer layers rather than heavy model re-engineering.

3. High-Level Architecture Blueprint



Module Breakdown:

1. **Local SLM Engine:** Runs sub-4B quantized models offline (Llama-3.2-3B, Qwen-2.5-Coder-3B) via llama.cpp or Ollama.
2. **Schema Enforcement Layer:** Enforces strict Pydantic schemas so model outputs conform strictly to executable JSON tools.
3. **Safety & Policy Gatekeeper:** Classifies extracted tool calls by risk tier (*Low*: read-only system stats; *Medium*: launch app/adjust volume; *High*: file deletion/system config changes). Enforces Human-in-the-Loop (HITL) authorization for high-risk commands.
4. **OS Execution Driver:** Python subprocess wrappers (os, psutil) executing approved JSON functions.

4. Target Evaluation Metrics & Testing Scope

To evaluate system performance and safety for the project defense, testing will measure three primary dimensions across a 50-command test suite:

1. **Schema & Intent Parsing Accuracy:** Percentage of natural language inputs correctly mapped to valid Pydantic JSON schemas.

2. **Execution Latency:** Total response time (Speech-to-Text + SLM Inference + OS Execution) measured locally in seconds.

3. **Safety Gatekeeper Interception Rate:** Percentage of high-risk/destructive actions correctly caught and routed to the Human-in-the-Loop (HITL) prompt.